Secure Software Design Development

Lab Midterm

Mr Muhammad Ahmad Nawaz

Presented by:

Muhammad Awais SP23-BCT-008

Muhammad Umair SP23-BCT-038

Muhammad Umar SP23-BCT-040

Date: 10th April 2026

Contents

Introduction: 3

Developing and Intentionally insecure App 3

Project Structure 3

App.py 4

Dashboard.html 4

Login.html 5

Result.html 5

Requirement.txt 5

Run the App (Demo in Video) 5

Push into the github for rest of the teammates do their part 6

Pushed to github successfully 6

VULNERABILITIES in the System (Demonstrated in Video) 7

1. IDOR ((Demonstrated in Video)) 7

2. CSRF (Demonstrated in Video) 7

3. Clickjacking (Demonstrated in Video) 7

Remediation and Improvement of GUI 8

App.py 8

Result.html 10

Login.html 11

Dashboard.html 12

Run the application and test for vulnerabilities. 13

Application after the remediation are demonstrated in the video. 13

CI/CD PIPELINE 13

Push into the Github 14

Pushed Successfully 14

Reports regarding the Project. 15

Final Look of the Repository 16

Link to Repository 16

Introduction:

This project presents the design, vulnerability analysis, and security hardening of a Student Result Portal developed as part of the CYC386 Secure Software Design and Development course at COMSATS University Islamabad. The portal, built using the Flask web framework in Python with an SQLite backend, serves as a realistic academic web application that allows students to view their semester results and administrators to manage and upload academic records. In its initial state, the application was deliberately engineered to contain three critical vulnerabilities drawn from the OWASP Top 10: Insecure Direct Object Reference (IDOR), Cross-Site Request Forgery (CSRF), and Clickjacking. Following the DevSecOps methodology, the team conducted a structured 48-hour security sprint encompassing Protection Needs Elicitation (PNE), STRIDE-based threat modeling, CVSS v3.1 risk assessment, secure code implementation, and automated security testing using both Static Application Security Testing (SAST) tools — Bandit and CodeQL — and Dynamic Application Security Testing (DAST) via OWASP ZAP, all integrated into a fully automated GitHub Actions CI/CD pipeline. The objective of this sprint was to simulate a real-world security engineering workflow, producing a deployable, attack-resistant application that demonstrates measurable improvement in security posture through evidence-based before-and-after analysis.

Developing and Intentionally insecure App

Project Structure

```
student-result-portal/
├── app.py
├── templates/
│     ├── login.html
│     ├── dashboard.html
│     ├── result.html
├── static/
├── requirements.txt
```

App.py

```python
from flask import Flask, render_template, request, redirect, session

app = Flask(__name__)
app.secret_key = "secret123"  # weak secret

# Fake database
users = {
    "student1": {"password": "123", "role": "student"},
    "admin": {"password": "admin", "role": "admin"}
}

results = {
    "1": {"name": "Ali", "marks": 85},
    "2": {"name": "Umar", "marks": 90}
}

@app.route('/', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']

        if username in users and users[username]["password"] == password:
            session['user'] = username
            return redirect('/dashboard')
    return render_template('login.html')


@app.route('/dashboard')
def dashboard():
    if 'user' not in session:
        return redirect('/')
    return render_template('dashboard.html', results=results)


# 🚨 IDOR VULNERABILITY
@app.route('/result/<id>')
def view_result(id):
    if 'user' not in session:
        return redirect('/')

    # ❌ No authorization check
    data = results.get(id)
    return render_template('result.html', data=data, id=id)


# 🚨 No CSRF Protection
@app.route('/edit/<id>', methods=['POST'])
def edit_result(id):
    new_marks = request.form['marks']
    results[id]['marks'] = new_marks
    return redirect('/dashboard')

if __name__ == '__main__':
    app.run(debug=True)
```

Dashboard.html

```html
<h2>Dashboard</h2>

{% for id, r in results.items() %}
    <p>
        {{ r.name }} - {{ r.marks }}
        <a href="/result/{{ id }}">View</a>
    </p>
{% endfor %}
```

Login.html

```html
<form method="POST">
    <input name="username" placeholder="Username"><br>
    <input name="password" type="password" placeholder="Password"><br>
    <button type="submit">Login</button>
</form>
```

Result.html

```html
<h2>Result</h2>

<p>Name: {{ data.name }}</p>
<p>Marks: {{ data.marks }}</p>

<form method="POST" action="/edit/{{ id }}">
    <input name="marks">
    <button type="submit">Update</button>
</form>
```

Requirement.txt

```
Flask
```

Run the App (Demo in Video)

```
┌──(venv)─(umar㉿shah)-[~/student-result-portal]
└─$ python app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a prod
uction WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 278-606-513
```

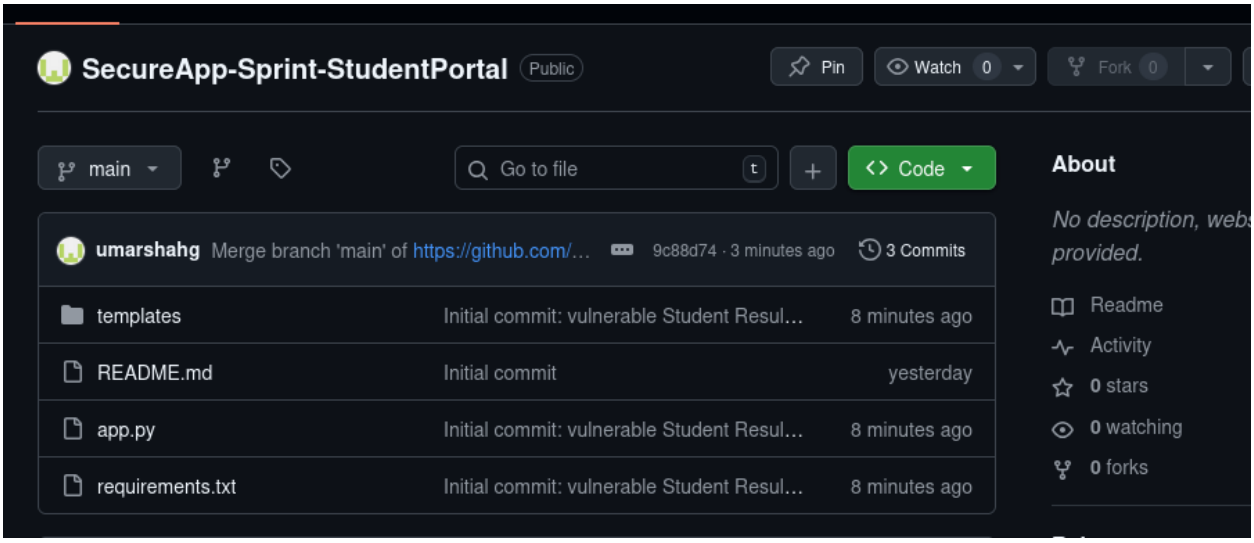
Push into the github for rest of the teammates do their part

```
┌──(venv)─(umar㉿shah)-[~/student-result-portal]
└─$ git init
Initialized empty Git repository in /home/umar/student-result-portal/.git/

┌──(venv)─(umar㉿shah)-[~/student-result-portal]
└─$ git add .

┌──(venv)─(umar㉿shah)-[~/student-result-portal]
└─$ git commit -m "Initial commit: vulnerable Student Result Portal setup"
[main (root-commit) 3929bff] Initial commit: vulnerable Student Result Portal setup
 5 files changed, 79 insertions(+)
 create mode 100644 app.py
 create mode 100644 requirements.txt
 create mode 100644 templates/dashboard.html
 create mode 100644 templates/login.html
 create mode 100644 templates/result.html
```

```
┌──(venv)─(umar㉿shah)-[~/student-result-portal]
└─$ git remote add origin https://github.com/umarshahg/SecureApp-Sprint-StudentPortal.git

┌──(venv)─(umar㉿shah)-[~/student-result-portal]
└─$ git branch -M main
```

```
┌──(venv)─(umar㉿shah)-[~/student-result-portal]
└─$ git push -u origin main
Username for 'https://github.com': umarshahg
Password for 'https://umarshahg@github.com':
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 4 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (10/10), 1.73 KiB | 196.00 KiB/s, done.
Total 10 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/umarshahg/SecureApp-Sprint-StudentPortal.git
   f381aaf..9c88d74  main → main
branch 'main' set up to track 'origin/main'.
```

Pushed to github successfully



VULNERABILITIES in the System (Demonstrated in Video)

1. IDOR ((Demonstrated in Video))

- /result/1 → change to /result/2
- Any student can see others' data

IDOR exists because the application does not verify whether the logged-in user is authorized to access the requested object (student result). By modifying the object ID in the URL, unauthorized data access is possible.

---

2. CSRF (Demonstrated in Video)

- No token → attacker can auto-submit form

CSRF vulnerability exists because the application does not verify the origin of requests. An attacker can craft a malicious page that triggers unintended actions when visited by an authenticated user.

---

3. Clickjacking (Demonstrated in Video)

- No headers → can be embedded in iframe

The application is vulnerable to clickjacking because it does not implement frame protection headers. This allows attackers to embed the application in an iframe and trick users into unintended actions.

Remediation and Improvement of GUI

App.py

```python
from flask import Flask, render_template, request, redirect, session
from flask_wtf import CSRFProtect, FlaskForm
from wtforms import StringField, SubmitField
from wtforms.validators import DataRequired

app = Flask(__name__)
app.secret_key = "supersecretkey123"

# Enable CSRF
csrf = CSRFProtect(app)

# Fake database
users = {
    "student1": {"password": "123", "role": "student"},
    "admin": {"password": "admin", "role": "admin"}
}

results = {
    "1": {"name": "Ali", "marks": 85},
    "2": {"name": "Umar", "marks": 90}
}

user_result_map = {
    "student1": "1",
    "admin": "all"
}

# Form
class EditForm(FlaskForm):
    marks = StringField('Marks', validators=[DataRequired()])
    submit = SubmitField('Update')


@app.route('/', methods=['GET', 'POST'])
@csrf.exempt
def login():
    error = None

    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']

        if username in users and users[username]["password"] == password:
            session['user'] = username
            return redirect('/dashboard')
        else:
            error = "Invalid Credentials"

    return render_template('login.html', error=error)
```

```python
@app.route('/dashboard')
def dashboard():
    if 'user' not in session:
        return redirect('/')
    return render_template('dashboard.html', results=results, user=session['user'])


@app.route('/result/<id>', methods=['GET', 'POST'])
def view_result(id):
    if 'user' not in session:
        return redirect('/')

    username = session['user']

    # IDOR FIX
    if username != "admin" and user_result_map.get(username) != id:
        return "Unauthorized Access", 403

    form = EditForm()


    # Only admin can edit
    if form.validate_on_submit():
        if username != "admin":
            return "Unauthorized Action", 403

        results[id]['marks'] = form.marks.data
        return redirect('/dashboard')



    return render_template('result.html', data=results[id], form=form)


@app.after_request
def security_headers(response):
    response.headers['X-Frame-Options'] = 'DENY'
    response.headers['Content-Security-Policy'] = "frame-ancestors 'none'"
    return response


if __name__ == '__main__':
    app.run(debug=True)
```

Result.html

```html
<!DOCTYPE html>
<html>
<head>
    <title>Result</title>
    <style>
        body {
            font-family: Arial;
            background: #eef2f3;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
        }
        .box {
            background: white;
            padding: 25px;
            border-radius: 10px;
            box-shadow: 0 0 10px gray;
            width: 300px;
        }
        input {
            width: 90%;
            padding: 8px;
            margin: 10px;
        }
        button {
            background: #007bff;
            color: white;
            border: none;
            padding: 8px;
            width: 95%;
        }
    </style>
</head>
<body>

<div class="box">
    <h2>{{ data.name }}</h2>
    <p>Current Marks: {{ data.marks }}</p>

    <form method="POST">
        {{ form.hidden_tag() }}
        {{ form.marks.label }} <br>
        {{ form.marks() }} <br>
        {{ form.submit() }}
    </form>
</div>

</body>
</html>
```

Login.html

```html
<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
    <style>
        body {
            font-family: Arial;
            background: linear-gradient(to right, #4facfe, #00f2fe);
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
        }
        .box {
            background: white;
            padding: 30px;
            border-radius: 10px;
            box-shadow: 0 0 10px gray;
            width: 300px;
            text-align: center;
        }
        input {
            width: 90%;
            padding: 10px;
            margin: 10px;
        }
        button {
            background: #4facfe;
            color: white;
            border: none;
            padding: 10px;
            width: 95%;
            cursor: pointer;
        }
        button:hover {
            background: #007bff;
        }
        .error {
            color: red;
        }
    </style>
</head>
<body>
    <div class="box">
        <h2>Student Portal</h2>
        {% if error %}
            <p class="error">{{ error }}</p>
        {% endif %}
        <form method="POST">
            <input name="username" placeholder="Username"><br>
            <input name="password" type="password" placeholder="Password"><br>
            <button>Login</button>
        </form>
    </div>
</body>
</html>
```

Dashboard.html

```html
<!DOCTYPE html>
<html>
<head>
    <title>Dashboard</title>
    <style>
        body {
            font-family: Arial;
            background: #f4f6f9;
            padding: 20px;
        }
        h2 {
            text-align: center;
        }
        .card {
            background: white;
            padding: 15px;
            margin: 10px auto;
            width: 300px;
            border-radius: 10px;
            box-shadow: 0 0 5px gray;
            text-align: center;
        }
        a {
            text-decoration: none;
            color: white;
            background: #28a745;
            padding: 5px 10px;
            border-radius: 5px;
        }
    </style>
</head>
<body>

<h2>Welcome {{ user }}</h2>

{% for id, r in results.items() %}
    <div class="card">
        <h3>{{ r.name }}</h3>
        <p>Marks: {{ r.marks }}</p>
        <a href="/result/{{ id }}">View</a>
    </div>
{% endfor %}

</body>
</html>
```

Run the application and test for vulnerabilities.

```
┌──(awais㉿awais)-[~]
└─$ ls
my_web_app

┌──(awais㉿awais)-[~]
└─$ cd my_web_app

┌──(awais㉿awais)-[~/my_web_app]
└─$ ls
app.py  templates

┌──(awais㉿awais)-[~/my_web_app]
└─$ cd templates

┌──(awais㉿awais)-[~/my_web_app/templates]
└─$ ls
dashboard.html  login.html  result.html
```

```
┌──(awais㉿awais)-[~/my_web_app]
└─$ python3 app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with watchdog (inotify)
 * Debugger is active!
 * Debugger PIN: 110-255-011
127.0.0.1 - - [10/Apr/2026 02:18:28] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [10/Apr/2026 02:18:29] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [10/Apr/2026 02:18:36] "POST / HTTP/1.1" 200 -
```

Application after the remediation are demonstrated in the video.

CI/CD PIPELINE

```
┌──(venv)─(umar㉿shah)-[~/student-result-portal]
└─$ mkdir -p .github/workflows

┌──(venv)─(umar㉿shah)-[~/student-result-portal]
└─$ nano .github/workflows/ci-cd.yml
```

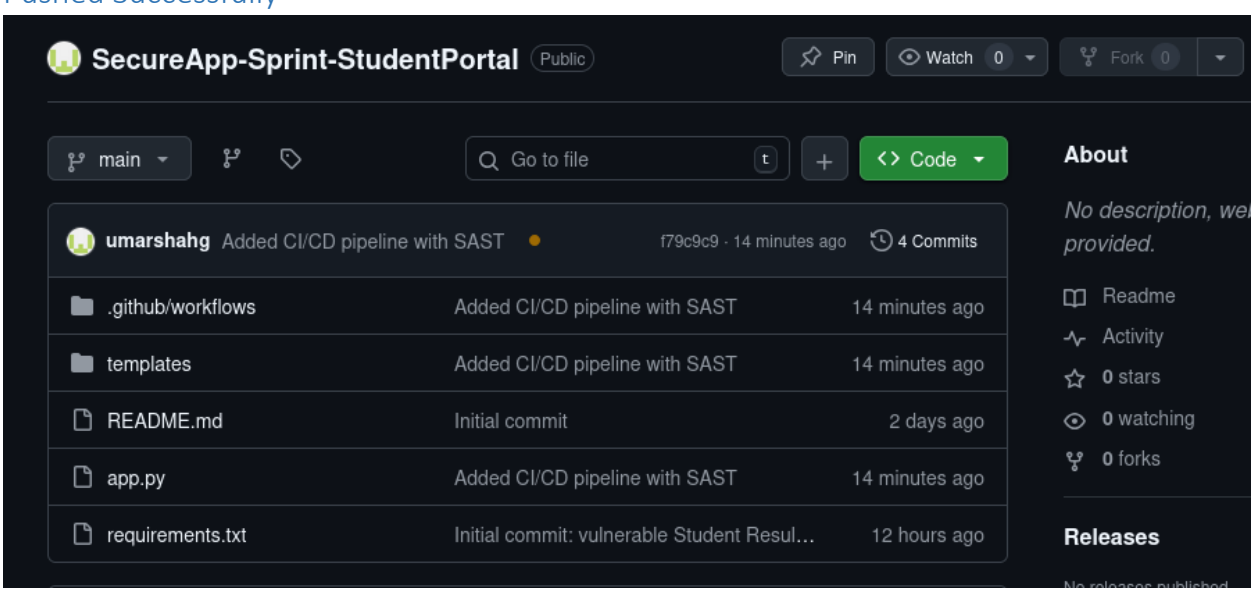
```yaml
name: Secure CI/CD Pipeline
on:
  push:
  pull_request:

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
    - name: Checkout Code
      uses: actions/checkout@v3

    - name: Setup Python
      uses: actions/setup-python@v4
      with:
        python-version: '3.10'

    - name: Install Dependencies
      run: |
        pip install -r requirements.txt

    - name: Run App Check
      run: |
        python -m py_compile app.py

  sast:
    name: SAST Scan (CodeQL)
    runs-on: ubuntu-latest
    permissions:
      security-events: write

    steps:
    - name: Checkout
      uses: actions/checkout@v3

    - name: Initialize CodeQL
      uses: github/codeql-action/init@v2
      with:
        languages: python

    - name: Autobuild
      uses: github/codeql-action/autobuild@v2

    - name: Perform CodeQL Analysis
      uses: github/codeql-action/analyze@v2
```

Push into the Github

```
┌──(venv)─(umar㉿shah)-[~/student-result-portal]
└─$ git add .

┌──(venv)─(umar㉿shah)-[~/student-result-portal]
└─$ git commit -m "Added CI/CD pipeline with SAST"
[main f79c9c9] Added CI/CD pipeline with SAST
 5 files changed, 244 insertions(+), 30 deletions(-)
 create mode 100644 .github/workflows/ci-cd.yml
```

```
┌──(venv)─(umar㉿shah)-[~/student-result-portal]
└─$ git push
Username for 'https://github.com': umarshahg
Password for 'https://umarshahg@github.com':
Enumerating objects: 16, done.
Counting objects: 100% (16/16), done.
Delta compression using up to 4 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (10/10), 3.01 KiB | 514.00 KiB/s, done.
Total 10 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), done.
To https://github.com/umarshahg/SecureApp-Sprint-StudentPortal.git
   9c88d74..f79c9c9  main → main
```

Pushed Successfully



Reports regarding the Project.

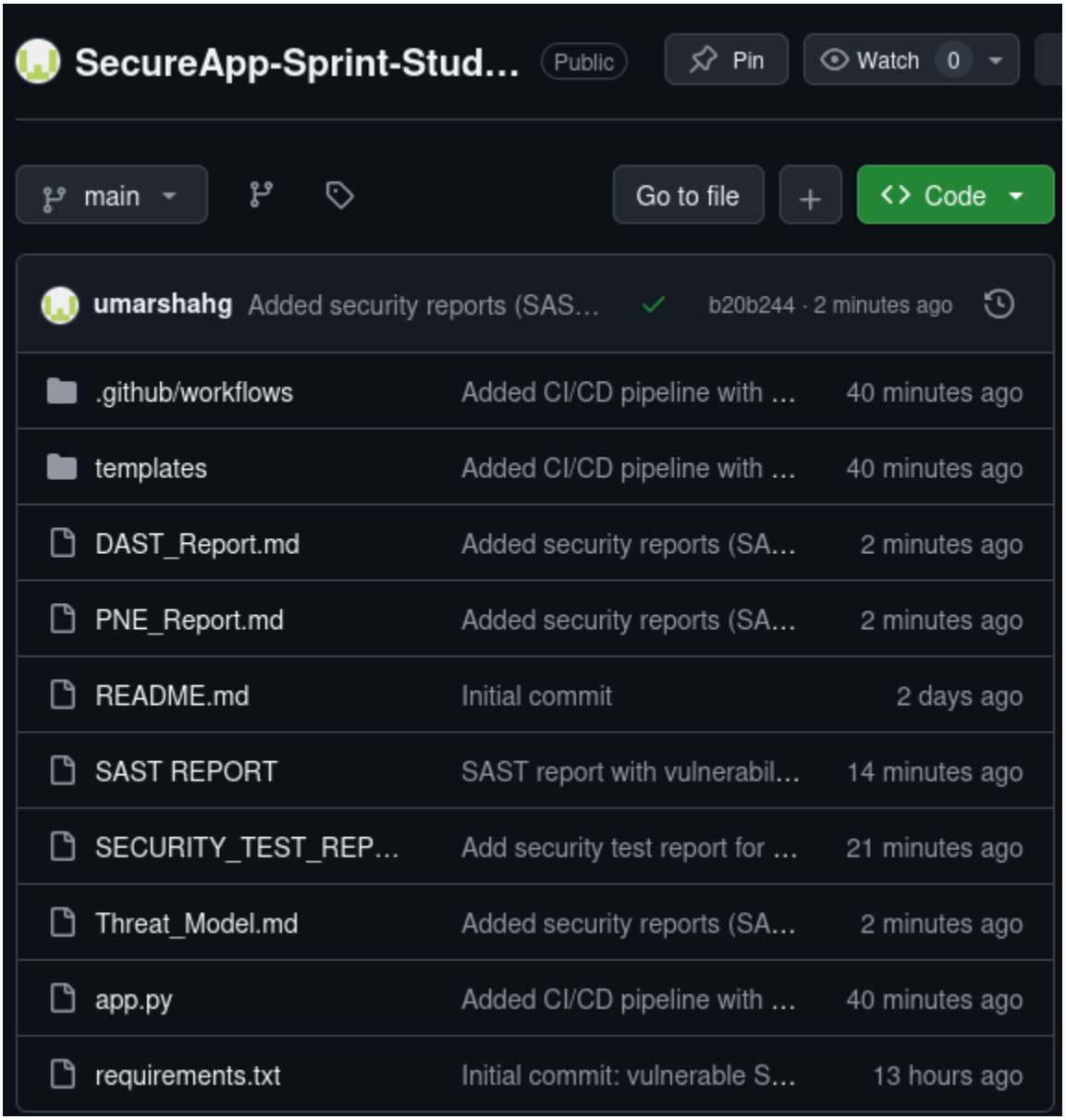
```
v)-(umair㉿nazir)-[~/student-result-portal]
$ nano DAST_Report.md

v)-(umair㉿nazir)-[~/student-result-portal]
$ nano PNE_Report.md

v)-(umair㉿nazir)-[~/student-result-portal]
$ nano Threat_Model.md

v)-(umair㉿nazir)-[~/student-result-portal]
$ nano Threat_Model.md

v)-(umair㉿nazir)-[~/student-result-portal]
$ git add .
```

(All the reports are available in the repository)

```
env)-(umair@nazir)-[~/student-result-portal]
git push
ername for 'https://github.com': umairshahg
ssword for 'https://umairshahg@github.com':
umerating objects: 6, done.
unting objects: 100% (6/6), done.
lta compression using up to 4 threads
mpressing objects: 100% (5/5), done.
tal 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
note: Resolving deltas: 100% (1/1), completed with 1 local object.
 https://github.com/umairshahg/SecureApp-Sprint-StudentPortal.git
561cd97..b20b244   main → main
```

\

Final Look of the Repository



Link to Repository

https://github.com/umarshahg/SecureApp-Sprint-StudentPortal